

Module 7 — Automation

Lesson 7.1

Headless mode

claude -p "..." is non-interactive Claude Code. It's the simplest way to wire Claude into a script, pipe, or job.

Why it matters

The interactive UI is great for humans. For scripts, cron jobs, hooks, and CI, you want a process that takes input, does the work, and exits. Headless mode is exactly that — and it inherits all your project's settings, MCP servers, and tools.

Basic usage

```
claude -p "summarize the changes in the last commit"
```

Behavior:

- Loads CLAUDE.md, settings.json, MCP servers — same as interactive
- Runs through the task in one shot
- Streams output to stdout
- Exits when done (or on error)

Useful flags

```
# Machine-readable output
claude -p "..." --output-format json

# Pipe input
git log -1 --format=%B | claude -p "rewrite this commit message for clarity"

# Pin model
claude -p "..." --model claude-haiku-4-5

# Specific permission mode (no prompts)
claude -p "..." --permission-mode acceptEdits

# Add a directory to the context
claude -p "..." --add-dir ../other-repo

# Hard skip permissions (sandboxes only!)
claude -p "..." --dangerously-skip-permissions
```

Check `claude --help` for the current full set.

Exit codes

Code	Meaning
0	Success
Non-zero	Failure (model error, tool error, permission denied, network)

In CI / scripts, check the exit code. Don't assume success from the absence of obvious error text.

Stdin and stdout

Stdin is treated as part of the user prompt. This makes piping natural:

```
cat error.log | claude -p "what's the most common failure mode in these logs?"
```

Stdout is the model's response. With `--output-format json`, it's a structured JSON blob with messages, usage, and metadata.

```
claude -p "list 3 ideas for X" --output-format json | jq '.messages[-1].content[0].text'
```

Settings inheritance

Headless mode loads the same hierarchy as interactive:

- 1 `~/.claude/settings.json`
- 2 `<cwd>/.claude/settings.json`
- 3 `<cwd>/.claude/settings.local.json`

CLAUDE.md is loaded from the cwd as usual.

MCP servers configured in settings are started. **They die when the headless invocation exits.** If your MCP server has slow startup, every headless run pays that cost. For high-frequency automation, this can dominate runtime — consider HTTP MCP servers or the SDK instead.

When to pick `claude -p` vs. the SDK

Situation	Pick
One-line shell integration	<code>claude -p</code>
Cron job, scheduled task	<code>claude -p</code> (simpler)
Service / long-running process	SDK (no subprocess per request)
You need fine programmatic control over the event stream	SDK
You need custom permission gating	SDK
You want it to "just work" in CI	<code>claude -p</code>

`claude -p` is fast to set up; the SDK is more flexible. Start with `claude -p`; graduate to the SDK when you need it.

Patterns

Generate a commit message

```
git add -p
diff=$(git diff --cached)
claude -p "Write a conventional-commit message for this diff:\n\n$diff"
```

(In practice, wrap in a function and bind to a key in your editor.)

Triage every new GitHub issue

```
gh issue list --state open --limit 5 --json number,title,body | \
  claude -p "For each issue, suggest one label from: bug, feature, docs, question. Reply as
  JSON array."
```

Summarize errors from yesterday's logs

```
cat /var/log/app.log | grep ERROR | claude -p "Top 3 distinct error patterns, with counts."
```

Generate release notes from a tag range

```
git log v1.2.0..HEAD --oneline | claude -p "Group into Features, Fixes, Refactors. Markdown."
```

Caching and cost

Headless invocations don't benefit from cross-run caching the way an interactive session does — each call is a fresh process. For high-frequency automation, consider:

- The SDK (longer-lived process, better cache hits)
- Aggregating many tasks into one larger call
- Pinning a small model where it's enough

Try it

- 1 `git log -1 --format=%B | claude -p "rewrite this commit message".` Pipe → process → output.
- 2 `claude -p "what tests should I add to this PR?" --output-format json.` Inspect the JSON.
- 3 Wire `claude -p` into a `git prepare-commit-msg` hook that suggests a commit message based on the staged diff.

Gotchas

- **Quoting on Windows shells.** Use double quotes; escape inner quotes carefully. PowerShell handles things differently than `cmd`.
- ``claude -p`` in CI hangs on permission prompts by default. Set `--permission-mode acceptEdits` or use a sandbox.
- **Stdin redirection** can include extra newlines. If your tool needs precise input, normalize first.
- **Long-running headless** invocations have no progress UI. Add your own (`tee` the output, log to file) for visibility.

Further reading

- 7.2 GitHub Actions — `claude -p` in CI
- 7.3 Scheduled and event-driven agents — cron, webhooks
- 6.1 SDK overview — when to upgrade from `-p`

Next

→ 7.2 Claude Code in GitHub Actions